LucidLink Connect — Integration Guide

A system administrator's guide to importing S3 objects into LucidLink filespaces using the LucidLink LucidAPI (ExternalDatastore / ExternalEntries).

Table of Contents

---

1. Overview

The LucidLink LucidAPI allows you to register an S3 bucket as an **ExternalDatastore** and then create **ExternalEntries** that reference individual S3 objects. These entries appear as normal files inside the LucidLink filespace but are *lazy-loaded* — no data is copied at import time. When a user opens a file, LucidLink streams it directly from S3.

Base URL pattern:

```
https://<lucid-api-host>/api/v1
```

All examples below use `$API_BASE` as the base URL.

Authentication: All API calls require a Bearer token in the `Authorization` header.

---

2. Prerequisites

Requirement	Details
LucidLink LucidAPI access	A valid API host URL and Bearer token
Filespace	An existing LucidLink filespace

Requirement	Details
S3 bucket	An accessible S3 bucket with objects to import
IAM credentials	`AWS_ACCESS_KEY_ID` and `AWS_SECRET_ACCESS_KEY` with `s3:ListBucket`, `s3:GetObject` permissions
Python 3.10+	For the code examples (uses `httpx` and `boto3`)

Install dependencies:

```
pip install httpx boto3
```

---

3. Step 1 — Authenticate with the LucidLink LucidAPI

All requests use Bearer token authentication. Obtain your token from the LucidLink Admin Portal or your organization's identity provider.

Header format:

```
Authorization: Bearer <your-token>
```

Python setup:

```
import httpx

API_BASE = "https://lucid-api.example.com/api/v1"
TOKEN = "your-bearer-token-here"

HEADERS = {
    "Authorization": f"Bearer {TOKEN}",
    "Content-Type": "application/json",
    "Accept": "application/json",
}

# Reusable async client with connection pooling
client = httpx.AsyncClient(
    timeout=httpx.Timeout(30.0),
    limits=httpx.Limits(max_connections=100,
max_keepalive_connections=50),
)
```

cURL equivalent:

```
# All subsequent curl examples assume these variables are set:
API_BASE="https://lucid-api.example.com/api/v1"
TOKEN="your-bearer-token-here"
```

---

4. Step 2 — List Filespaces

Retrieve all filespaces associated with your token.

API

```
GET /filespaces
```

Python

```
async def list_filespaces():
    response = await client.get(
        f"{API_BASE}/filespaces",
        headers=HEADERS,
        timeout=10.0,
    )
    response.raise_for_status()
    data = response.json()

    # Response may be a list or wrapped in {"data": [...]}
    filespaces = data if isinstance(data, list) else data.get("data", [])

    for fs in filespaces:
        print(f"  {fs['name']}  (id: {fs['id']})")

    return filespaces
```

cURL

```
curl -s -H "Authorization: Bearer $TOKEN" \
     -H "Accept: application/json" \
     "$API_BASE/filespaces" | jq '.data[] | {name, id}'
```

Response

```
{
  "data": [
    {
```

```
      "id": "a1b2c3d4-5678-9abc-def0-111111111111",
      "name": "production-filespace"
    },
    {
      "id": "b2c3d4e5-6789-abcd-ef01-222222222222",
      "name": "staging-filespace"
    }
  ]
}
```

Choose a filespace and save its `id` for subsequent calls.

```
FILESPACE_ID = "a1b2c3d4-5678-9abc-def0-111111111111"
```

---

5. Step 3 — List ExternalDatastores

List the Datastores already configured for a filespace.

API

```
GET /filespaces/{filespace_id}/external/data-stores
```

Python

```python
async def list_datastores(filespace_id: str):
    response = await client.get(
        f"{API_BASE}/filespaces/{filespace_id}/external/data-stores",
        headers=HEADERS,
        timeout=10.0,
    )
    response.raise_for_status()
    data = response.json()

    datastores = data if isinstance(data, list) else data.get("data", [])

    for ds in datastores:
        bucket = ds.get("s3StorageParams", {}).get("bucketName", "N/A")
        print(f"  {ds['name']}  (id: {ds['id']}, bucket: {bucket})")

    return datastores
```

cURL

```
curl -s -H "Authorization: Bearer $TOKEN" \
     -H "Accept: application/json" \
     "$API_BASE/filespaces/$FILESPACE_ID/external/data-stores" \
     | jq '.data[] | {id, name, bucket: .s3StorageParams.bucketName}'
```

Response

```
{
  "data": [
    {
      "id": "ds-uuid-1234",
      "name": "Media Archive",
      "kind": "S3DataStore",
      "s3StorageParams": {
        "bucketName": "my-media-bucket",
        "region": "us-east-1",
        "endpoint": "https://s3.amazonaws.com",
        "useVirtualAddressing": true,
        "urlExpirationMinutes": 10080
      }
    }
  ]
}
```

If no Datastores exist yet, proceed to Step 4 to create one.

---

6. Step 4 — Create an S3 Datastore

Register an S3 bucket as an ExternalDatastore for a filespace. This requires IAM credentials with access to the bucket.

API

```
POST /filespaces/{filespace_id}/external/data-stores
```

Request Body

```
{
  "name": "My S3 Datastore",
  "kind": "S3DataStore",
  "s3StorageParams": {
    "bucketName": "my-bucket-name",
    "accessKey": "AKIAIOSFODNN7EXAMPLE",
    "secretKey": "wJalrXUtnFEMI/K7MDENG/bPxRfiCYEXAMPLEKEY",
```

```
    "region": "us-east-1",
    "endpoint": "https://s3.amazonaws.com",
    "useVirtualAddressing": true,
    "urlExpirationMinutes": 10080
  }
}
```

Field	Required	Description
`name`	Yes	Display name for the Datastore
`kind`	Yes	Must be `"S3DataStore"`
`bucketName`	Yes	S3 bucket name
`accessKey`	Yes	IAM access key ID
`secretKey`	Yes	IAM secret access key
`region`	No	AWS region (default: `us-east-1`)
`endpoint`	No	Custom S3-compatible endpoint URL
`useVirtualAddressing`	No	Virtual-hosted style URLs (default: `true`)
`urlExpirationMinutes`	No	Pre-signed URL TTL (default: `10080` = 7 days)

Python

```python
async def create_datastore(
    filespace_id: str,
    name: str,
    bucket: str,
    access_key: str,
    secret_key: str,
    region: str = "us-east-1",
    endpoint: str = "",
):
    payload = {
        "name": name,
        "kind": "S3DataStore",
        "s3StorageParams": {
            "bucketName": bucket,
            "accessKey": access_key,
            "secretKey": secret_key,
            "region": region,
            "useVirtualAddressing": True,
            "urlExpirationMinutes": 10080,
        },
    }
    if endpoint:
        payload["s3StorageParams"]["endpoint"] = endpoint
```

```
    response = await client.post(
        f"{API_BASE}/filespaces/{filespace_id}/external/data-stores",
        headers=HEADERS,
        json=payload,
    )
    response.raise_for_status()
    result = response.json()

    datastore_id = result.get("data", result).get("id")
    print(f"Created Datastore: {datastore_id}")
    return datastore_id
```

cURL

```
curl -s -X POST \
     -H "Authorization: Bearer $TOKEN" \
     -H "Content-Type: application/json" \
     "$API_BASE/filespaces/$FILESPACE_ID/external/data-stores" \
     -d '{
       "name": "My S3 Datastore",
       "kind": "S3DataStore",
       "s3StorageParams": {
         "bucketName": "my-bucket-name",
         "accessKey": "AKIAIOSFODNN7EXAMPLE",
         "secretKey": "wJalrXUtnFEMI/K7MDENG/bPxRfiCYEXAMPLEKEY",
         "region": "us-east-1",
         "useVirtualAddressing": true,
         "urlExpirationMinutes": 10080
       }
     }' | jq '.data.id'
```

---

7. Step 5 — Browse S3 Bucket Contents

Before importing, you typically want to browse the S3 bucket to identify which objects to import. This step uses **boto3 directly against S3** (not the LucidLink API).

Python

```
import boto3

def list_s3_objects(
    bucket: str,
    prefix: str = "",
    access_key: str = "",
    secret_key: str = "",
    region: str = "us-east-1",
    endpoint: str = "",
    max_items: int = 5000,
```

```
):
    """
    List S3 objects with folder-like navigation using the '/' delimiter.
    Returns folders (CommonPrefixes) and files (Contents).
    """
    session = boto3.Session(
        aws_access_key_id=access_key,
        aws_secret_access_key=secret_key,
        region_name=region,
    )
    client_kwargs = {}
    if endpoint:
        client_kwargs["endpoint_url"] = endpoint

    s3 = session.client("s3", **client_kwargs)

    folders = []
    files = []
    continuation_token = None

    while len(folders) + len(files) < max_items:
        params = {
            "Bucket": bucket,
            "Prefix": prefix,
            "Delimiter": "/",
            "MaxKeys": 1000,
        }
        if continuation_token:
            params["ContinuationToken"] = continuation_token

        resp = s3.list_objects_v2(**params)

        # Folders (common prefixes)
        for cp in resp.get("CommonPrefixes", []):
            folders.append({
                "key": cp["Prefix"],
                "name": cp["Prefix"].rstrip("/").rsplit("/", 1)[-1],
                "type": "folder",
            })

        # Files
        for obj in resp.get("Contents", []):
            if obj["Key"] == prefix:
                continue  # skip the prefix itself
            files.append({
                "key": obj["Key"],
                "name": obj["Key"].rsplit("/", 1)[-1],
                "type": "file",
                "size": obj["Size"],
                "last_modified": obj["LastModified"].isoformat(),
            })

        if not resp.get("IsTruncated"):
            break
```

```
        continuation_token = resp.get("NextContinuationToken")

    # Sort: folders first (alpha), then files (alpha)
    folders.sort(key=lambda x: x["name"].lower())
    files.sort(key=lambda x: x["name"].lower())

    return folders + files
```

cURL (AWS CLI)

```
# List top-level contents
aws s3api list-objects-v2 \
    --bucket my-bucket-name \
    --delimiter "/" \
    --prefix "" \
    --max-keys 100 | jq '{folders: .CommonPrefixes, files: .Contents}'

# Navigate into a folder
aws s3api list-objects-v2 \
    --bucket my-bucket-name \
    --delimiter "/" \
    --prefix "photos/2025/" \
    --max-keys 100
```

---

8. Step 6 — Prepare Folder Structure in the Filespace

Before importing files, ensure the target directory structure exists in the filespace. The LucidLink API requires creating folders explicitly before placing files in them.

API — Resolve Entry by Path

```
GET /filespaces/{filespace_id}/entries/resolve?path={path}
```

Returns the entry ID for an existing path.

API — Create Folder

```
POST /filespaces/{filespace_id}/entries
```

```
{
  "name": "folder-name",
  "type": "dir",
```

```
  "parentId": "parent-entry-uuid"
}
```

Python

```python
async def resolve_entry_path(filespace_id: str, path: str) -> str | None:
    """Resolve a filespace path to its entry ID. Returns None if not
found."""
    response = await client.get(
        f"{API_BASE}/filespaces/{filespace_id}/entries/resolve",
        headers=HEADERS,
        params={"path": path},
    )
    if response.status_code == 200:
        data = response.json()
        return data.get("data", data).get("id")
    return None


async def create_folder(filespace_id: str, name: str, parent_id: str =
None) -> str:
    """Create a folder and return its entry ID."""
    payload = {"name": name, "type": "dir"}
    if parent_id:
        payload["parentId"] = parent_id

    response = await client.post(
        f"{API_BASE}/filespaces/{filespace_id}/entries",
        headers=HEADERS,
        json=payload,
    )
    response.raise_for_status()
    data = response.json()
    return data.get("data", data).get("id")


async def ensure_folder_structure(filespace_id: str, s3_key: str):
    """
    Given an S3 key like "bucket-name/data/2025/report.csv",
    create the folder tree:  /bucket-name/data/2025/

    Returns the last folder's entry ID.
    """
    parts = s3_key.split("/")
    # Remove the filename (last element)
    folder_parts = parts[:-1] if parts[-1] else parts[:-2]

    current_path = ""
    parent_id = None

    # Resolve root
```

```
    root_id = await resolve_entry_path(filespace_id, "/")
    parent_id = root_id

    for folder_name in folder_parts:
        current_path += f"/{folder_name}"

        # Try to resolve existing folder
        existing_id = await resolve_entry_path(filespace_id, current_path)
        if existing_id:
            parent_id = existing_id
            continue

        # Create the folder
        try:
            folder_id = await create_folder(filespace_id, folder_name,
parent_id)
            parent_id = folder_id
        except httpx.HTTPStatusError as e:
            if e.response.status_code == 409:
                # Race condition — folder was created by another process
                existing_id = await resolve_entry_path(filespace_id,
current_path)
                if existing_id:
                    parent_id = existing_id
                    continue
            raise

    return parent_id
```

cURL

```
# Resolve the root entry ID
ROOT_ID=$(curl -s -H "Authorization: Bearer $TOKEN" \
    "$API_BASE/filespaces/$FILESPACE_ID/entries/resolve?path=/" \
    | jq -r '.data.id')

# Create a folder at root level
curl -s -X POST \
    -H "Authorization: Bearer $TOKEN" \
    -H "Content-Type: application/json" \
    "$API_BASE/filespaces/$FILESPACE_ID/entries" \
    -d "{\"name\": \"my-bucket\", \"type\": \"dir\", \"parentId\":
\"$ROOT_ID\"}" \
    | jq '.data.id'
```

---

9. Step 7 — Import S3 Objects as ExternalEntries

This is the core operation — creating ExternalEntries that map S3 objects into the filespace.

API

```
POST /filespaces/{filespace_id}/external/entries
```

Request Body

```
{
  "path": "/bucket-name/path/to/file.ext",
  "kind": "SingleObjectFile",
  "dataStoreId": "ds-uuid-1234",
  "singleObjectFileParams": {
    "objectId": "path/to/file.ext"
  }
}
```

Field	Description
`path`	Full target path in the filespace (must start with `/`)
`kind`	Must be `"SingleObjectFile"`
`dataStoreId`	UUID of the Datastore created in Step 4
`singleObjectFileParams.objectId`	The S3 object key (path within the bucket)

Python — Single File Import

```
async def import_file(
    filespace_id: str,
    datastore_id: str,
    s3_key: str,
    bucket_name: str,
):
    """
    Import a single S3 object into the filespace.

    The file will appear at: /{bucket_name}/{s3_key}
    For example, S3 key "photos/image.jpg" in bucket "media"
    becomes /media/photos/image.jpg in the filespace.
    """
    ll_path = f"/{bucket_name}/{s3_key}"

    payload = {
        "path": ll_path,
        "kind": "SingleObjectFile",
        "dataStoreId": datastore_id,
        "singleObjectFileParams": {
            "objectId": s3_key,
```

```
        },
    }

    response = await client.post(
        f"{API_BASE}/filespaces/{filespace_id}/external/entries",
        headers=HEADERS,
        json=payload,
    )

    if response.status_code in (200, 201):
        print(f"  Imported: {ll_path}")
        return "created"
    elif response.status_code == 409:
        print(f"  Skipped (already exists): {ll_path}")
        return "skipped"
    elif response.status_code == 400:
        body = response.text
        if "already exists" in body.lower():
            print(f"  Skipped (already exists): {ll_path}")
            return "skipped"
        raise Exception(f"Import failed (400): {body[:500]}")
    else:
        raise Exception(f"Import failed ({response.status_code}):
{response.text[:500]}")
```

Python — Batch Folder Import

```
import asyncio

async def import_folder(
    filespace_id: str,
    datastore_id: str,
    bucket_name: str,
    prefix: str,
    access_key: str,
    secret_key: str,
    region: str = "us-east-1",
    batch_size: int = 25,
):
    """
    Import all S3 objects under a prefix into the filespace.
    Creates folder structure automatically.
    """
    # 1. List all objects under the prefix (recursive, no delimiter)
    s3 = boto3.client(
        "s3",
        aws_access_key_id=access_key,
        aws_secret_access_key=secret_key,
        region_name=region,
    )
    paginator = s3.get_paginator("list_objects_v2")
```

```
    keys = []
    for page in paginator.paginate(Bucket=bucket_name, Prefix=prefix):
        for obj in page.get("Contents", []):
            key = obj["Key"]
            if not key.endswith("/"):  # skip folder markers
                keys.append(key)

    print(f"Found {len(keys)} objects to import")

    # 2. Pre-create all unique folder paths
    unique_dirs = set()
    for key in keys:
        parts = key.rsplit("/", 1)
        if len(parts) > 1:
            unique_dirs.add(parts[0])

    for dir_path in sorted(unique_dirs):
        full_key = f"{bucket_name}/{dir_path}/placeholder"
        await ensure_folder_structure(filespace_id, full_key)

    # 3. Import files in parallel batches
    created = 0
    skipped = 0
    failed = 0

    for i in range(0, len(keys), batch_size):
        batch = keys[i : i + batch_size]
        tasks = [
            import_file(filespace_id, datastore_id, key, bucket_name)
            for key in batch
        ]
        results = await asyncio.gather(*tasks, return_exceptions=True)

        for result in results:
            if isinstance(result, Exception):
                failed += 1
                print(f"  Error: {result}")
            elif result == "created":
                created += 1
            elif result == "skipped":
                skipped += 1

        # Small delay between batches to avoid rate limiting
        await asyncio.sleep(0.05)

    print(f"\nImport complete: {created} created, {skipped} skipped,
{failed} failed")
```

cURL — Single File

```
DATASTORE_ID="ds-uuid-1234"
BUCKET="my-bucket-name"
S3_KEY="photos/2025/image.jpg"

curl -s -X POST \
     -H "Authorization: Bearer $TOKEN" \
     -H "Content-Type: application/json" \
     "$API_BASE/filespaces/$FILESPACE_ID/external/entries" \
     -d "{
       \"path\": \"/$BUCKET/$S3_KEY\",
       \"kind\": \"SingleObjectFile\",
       \"dataStoreId\": \"$DATASTORE_ID\",
       \"singleObjectFileParams\": {
         \"objectId\": \"$S3_KEY\"
       }
     }"
```

---

10. Complete Workflow Example

A self-contained script that performs the entire workflow end-to-end.

```
#!/usr/bin/env python3
"""
LucidLink ExternalDatastore — Complete Import Workflow

Imports all objects from an S3 prefix into a LucidLink filespace.
"""

import asyncio
import boto3
import httpx

# ── Configuration ──────────────────────────────────────────────
API_BASE      = "https://lucid-api.example.com/api/v1"
TOKEN         = "your-bearer-token"
AWS_ACCESS    = "AKIAIOSFODNN7EXAMPLE"
AWS_SECRET    = "wJalrXUtnFEMI/K7MDENG/bPxRfiCYEXAMPLEKEY"
AWS_REGION    = "us-east-1"
BUCKET_NAME   = "my-media-bucket"
IMPORT_PREFIX = "videos/2025/"   # S3 prefix to import (empty for entire
bucket)
# ──────────────────────────────────────────────────────────────

HEADERS = {
    "Authorization": f"Bearer {TOKEN}",
    "Content-Type": "application/json",
    "Accept": "application/json",
}
```

```
async def main():
    async with httpx.AsyncClient(timeout=30.0) as client:

        # ── Step 1: List filespaces ──
        print("Fetching filespaces...")
        resp = await client.get(f"{API_BASE}/filespaces", headers=HEADERS)
        resp.raise_for_status()
        filespaces = resp.json().get("data", resp.json())

        print("Available filespaces:")
        for i, fs in enumerate(filespaces):
            print(f"  [{i}] {fs['name']}  ({fs['id']})")

        # Select filespace (hardcoded index for automation, or prompt
user)
        filespace = filespaces[0]
        fs_id = filespace["id"]
        print(f"\nUsing filespace: {filespace['name']}")

        # ── Step 2: List or create Datastore ──
        print("\nFetching Datastores...")
        resp = await client.get(
            f"{API_BASE}/filespaces/{fs_id}/external/data-stores",
            headers=HEADERS,
        )
        resp.raise_for_status()
        datastores = resp.json().get("data", resp.json())

        # Find existing Datastore for our bucket, or create one
        ds_id = None
        for ds in datastores:
            if ds.get("s3StorageParams", {}).get("bucketName") ==
BUCKET_NAME:
                ds_id = ds["id"]
                print(f"Found existing Datastore: {ds['name']} ({ds_id})")
                break

        if not ds_id:
            print(f"Creating Datastore for bucket '{BUCKET_NAME}'...")
            resp = await client.post(
                f"{API_BASE}/filespaces/{fs_id}/external/data-stores",
                headers=HEADERS,
                json={
                    "name": f"S3 — {BUCKET_NAME}",
                    "kind": "S3DataStore",
                    "s3StorageParams": {
                        "bucketName": BUCKET_NAME,
                        "accessKey": AWS_ACCESS,
                        "secretKey": AWS_SECRET,
                        "region": AWS_REGION,
                        "useVirtualAddressing": True,
                        "urlExpirationMinutes": 10080,
                    },
```

```
                },
            )
            resp.raise_for_status()
            ds_id = resp.json().get("data", resp.json()).get("id")
            print(f"Created Datastore: {ds_id}")

        # ── Step 3: List S3 objects ──
        print(f"\nScanning S3 bucket '{BUCKET_NAME}' prefix
'{IMPORT_PREFIX}'...")
        s3 = boto3.client(
            "s3",
            aws_access_key_id=AWS_ACCESS,
            aws_secret_access_key=AWS_SECRET,
            region_name=AWS_REGION,
        )
        paginator = s3.get_paginator("list_objects_v2")
        keys = []
        for page in paginator.paginate(Bucket=BUCKET_NAME,
Prefix=IMPORT_PREFIX):
            for obj in page.get("Contents", []):
                if not obj["Key"].endswith("/"):
                    keys.append(obj["Key"])

        print(f"Found {len(keys)} objects to import")
        if not keys:
            print("Nothing to import.")
            return

        # ── Step 4: Create folder structure ──
        print("\nCreating folder structure...")
        unique_dirs = sorted(set(
            k.rsplit("/", 1)[0] for k in keys if "/" in k
        ))

        for dir_path in unique_dirs:
            parts = dir_path.split("/")
            current_path = ""
            parent_id = None

            # Resolve root
            r = await client.get(
                f"{API_BASE}/filespaces/{fs_id}/entries/resolve",
                headers=HEADERS,
                params={"path": "/"},
            )
            if r.status_code == 200:
                parent_id = r.json().get("data", r.json()).get("id")

            for folder_name in parts:
                current_path += f"/{folder_name}"
                r = await client.get(
                    f"{API_BASE}/filespaces/{fs_id}/entries/resolve",
                    headers=HEADERS,
                    params={"path":
```

```
f"/{BUCKET_NAME}/{current_path.lstrip('/')}"},
                )
                if r.status_code == 200:
                    parent_id = r.json().get("data", r.json()).get("id")
                    continue

                # Create folder
                r = await client.post(
                    f"{API_BASE}/filespaces/{fs_id}/entries",
                    headers=HEADERS,
                    json={"name": folder_name, "type": "dir", "parentId":
parent_id},
                )
                if r.status_code in (200, 201):
                    parent_id = r.json().get("data", r.json()).get("id")
                elif r.status_code == 409:
                    # Already exists (race condition) — resolve it
                    r2 = await client.get(
                        f"{API_BASE}/filespaces/{fs_id}/entries/resolve",
                        headers=HEADERS,
                        params={"path":
f"/{BUCKET_NAME}/{current_path.lstrip('/')}"},
                    )
                    if r2.status_code == 200:
                        parent_id = r2.json().get("data",
r2.json()).get("id")

        # ── Step 5: Import files ──
        print(f"\nImporting {len(keys)} files...")
        created = skipped = failed = 0
        batch_size = 25

        for i in range(0, len(keys), batch_size):
            batch = keys[i : i + batch_size]

            async def do_import(s3_key):
                ll_path = f"/{BUCKET_NAME}/{s3_key}"
                r = await client.post(
                    f"{API_BASE}/filespaces/{fs_id}/external/entries",
                    headers=HEADERS,
                    json={
                        "path": ll_path,
                        "kind": "SingleObjectFile",
                        "dataStoreId": ds_id,
                        "singleObjectFileParams": {"objectId": s3_key},
                    },
                )
                return r.status_code, r.text

            results = await asyncio.gather(
                *[do_import(key) for key in batch],
                return_exceptions=True,
            )
```

```
            for key, result in zip(batch, results):
                if isinstance(result, Exception):
                    failed += 1
                    print(f"  ERROR  {key}: {result}")
                else:
                    status, body = result
                    if status in (200, 201):
                        created += 1
                    elif status == 409 or (status == 400 and "already" in
body.lower()):
                        skipped += 1
                    else:
                        failed += 1
                        print(f"  FAIL   {key}: HTTP {status}")

            # Progress
            done = min(i + batch_size, len(keys))
            print(f"  Progress: {done}/{len(keys)}")
            await asyncio.sleep(0.05)

        print(f"\nDone! Created: {created}, Skipped: {skipped}, Failed:
{failed}")


if __name__ == "__main__":
    asyncio.run(main())
```

---

11. Error Handling Reference

HTTP Status	Meaning	Recommended Action
200 / 201	Success	Process response data
400	Bad request	Check payload format; if body contains "already exists", treat as skip
401	Unauthorized	Token is invalid or expired — re-authenticate
403	Forbidden	Insufficient permissions for this filespace/operation
404	Not found	Resource (filespace, Datastore, entry path) does not exist
409	Conflict	Entry or folder already exists — safe to skip or resolve existing
5xx	Server error	Retry with exponential backoff

Connection-level errors:

Error	Cause
Timeout	API did not respond within 30 seconds — retry or check API host
Connection refused	API host is unreachable — verify URL and network

Error	Cause
SSL error	Certificate issue — verify API host URL scheme

12. Key Concepts

Lazy Loading

ExternalEntries are **metadata references**, not copies. No S3 data is transferred during import. When a user opens the file through LucidLink, the client streams the object directly from S3 using pre-signed URLs generated by the Datastore configuration.

Path Mapping

Files are organized under the bucket name in the filespace:

```
S3:        s3://my-bucket/photos/2025/image.jpg
Filespace: /my-bucket/photos/2025/image.jpg
```

The `objectId` in the import payload is the raw S3 key (`photos/2025/image.jpg`), while `path` is the full filespace path with leading slash and bucket name prefix.

Idempotency

Importing the same S3 object twice returns `409 Conflict` or `400` with "already exists". Both are safe to treat as no-ops. This makes it safe to re-run imports without duplicating entries.

Rate Limiting

For bulk imports, batch concurrent requests (25 at a time is a reasonable default) with a small delay between batches (~50ms) to avoid overwhelming the API.

Folder Creation Order

Folders must be created top-down (parent before child) because each folder creation requires the parent's entry ID. When importing many files across different directory trees, collect all unique directory paths, sort them, and create them sequentially before starting file imports.

Datastore Credentials

The IAM credentials provided when creating a Datastore are stored by the LucidLink service. These credentials are used to generate pre-signed URLs when users access imported files. Ensure the IAM user has at minimum `s3:GetObject` permission on the bucket. For browsing, `s3:ListBucket` is also required.

Deleting a Datastore

```
DELETE /filespaces/{filespace_id}/external/data-stores/{datastore_id}
```

Returns `200`–`204` on success. Deleting a Datastore does **not** remove the ExternalEntries that reference it — those entries will become inaccessible until the Datastore is re-created or entries are cleaned up.